



# 1. Product Overview

## 1.1 Problem Statement

Huntington's Disease (HD) is a fatal neurodegenerative disorder caused by an autosomal dominant mutation — a CAG trinucleotide repeat expansion in the HTT gene. The disease is typically diagnosed only after noticeable motor and cognitive symptoms appear, by which time significant and irreversible neurodegeneration has already occurred.

Current diagnostic methods rely on single-modal approaches — MRI alone, genetic testing alone, or clinical observation alone — each of which provides an incomplete picture. No existing system integrates all three data modalities into a unified predictive framework for early-stage detection and longitudinal progression analysis.

## 1.2 Proposed Solution

NeuroSense is an AI-powered clinical decision support framework that integrates structural MRI, functional MRI (fMRI), genetic biomarkers (CAG repeat count), and UHDRS clinical scores to enable early HD detection and progression forecasting. The system is designed for use by neurologists and clinical researchers.

## 1.3 Key Differentiators

- Multi-modal fusion combining 3D neuroimaging, genetics, and clinical scores
- 3D ResNet-50 for volumetric brain feature extraction (not slice-based)
- Bi-LSTM encoder for longitudinal visit sequence modelling
- Cross-modal attention fusion (transformer-based)
- Dual explainability: GradCAM++ heatmaps on MRI + SHAP for tabular features
- Three-class HD staging: Pre-manifest / Early / Advanced
- 12- and 24-month disease progression trajectory forecasting
- FastAPI backend + React clinical dashboard

# 2. Goals and Success Metrics

## 2.1 Primary Goals

- Achieve  $AUC-ROC \geq 0.85$  for HD three-class staging on the PREDICT-HD test split
- Achieve  $F1-score \geq 0.80$  for early-stage HD vs pre-manifest discrimination
- Demonstrate that multi-modal fusion outperforms any single modality by  $\geq 5\%$  AUC
- Provide GradCAM++ heatmaps overlaid on axial MRI slices for every prediction
- Process a complete prediction (MRI + clinical data) in under 30 seconds on a GPU

## 2.2 Evaluation Metrics

| Metric | Target | Why It Matters |
|--------|--------|----------------|
|--------|--------|----------------|

|                       |                             |                                                            |
|-----------------------|-----------------------------|------------------------------------------------------------|
| AUC-ROC (per class)   | $\geq 0.85$                 | Primary classification metric; handles class imbalance     |
| F1-Score              | $\geq 0.80$                 | Balances precision and recall for minority classes         |
| MAE (progression)     | $\leq 3$ months             | Accuracy of 12/24-month trajectory forecast                |
| Calibration (ECE)     | $\leq 0.10$                 | Predicted probabilities should match empirical frequencies |
| Inference latency     | $\leq 30$ seconds           | Clinical usability threshold                               |
| Ablation $\Delta$ AUC | $\geq 5\%$ gain from fusion | Proves multi-modal fusion adds measurable value            |

## 3. Datasets

### 3.1 Primary Dataset: PREDICT-HD

- Source: Functional Imaging Biomedical Informatics Research Network (FITBIR) — [fitbir.nih.gov](http://fitbir.nih.gov)
- Content: Longitudinal MRI, cognitive assessments, motor scores (UHDRS), and CAG repeat counts for HD gene-expansion carriers and healthy controls
- Access: Requires data access agreement via FITBIR portal (allow 2–4 weeks for approval)
- Approximate cohort: ~1,000 subjects, multiple visits over 6+ years

### 3.2 Supplementary Dataset: OpenNeuro HD

- Dataset ID: ds004040 at [openneuro.org](http://openneuro.org) — publicly available, no approval required
- Use during development while PREDICT-HD access is pending
- Structural MRI in BIDS format, annotated with clinical labels

### 3.3 Data Split Strategy

- Split: 70% train / 15% validation / 15% test
- Stratified by disease stage (pre-manifest, early, advanced) to preserve class distribution
- No subject appears in more than one split (subject-level split, not scan-level)
- Report class distribution in each split to document any residual imbalance

### 3.4 Data Ethics

- All datasets are de-identified per HIPAA Safe Harbor standards
- Data stored only on local lab machines or institutional servers — never public cloud
- FITBIR data use agreement prohibits redistribution or re-identification attempts

## 4. System Architecture

### 4.1 Pipeline Overview

The NeuroSense pipeline has six sequential stages: (1) data acquisition and preprocessing, (2) modality-specific encoding, (3) cross-modal fusion, (4) disease staging classification, (5) progression forecasting, and (6) explainability generation and dashboard display.

### 4.2 Module Specifications

#### 4.2.1 Preprocessing Module

- MRI pipeline: skull stripping (FSL BET), bias field correction (FSL FAST), affine registration to MNI152 space (ANTs), resampling to 1mm isotropic, z-score normalisation per-subject

- Tabular pipeline: min-max normalisation for continuous features (age, UHDRS subscores), one-hot encoding for categorical features, median imputation for missing values
- Output: 3D NIfTI volume (96×96×96) + normalised tabular vector (length 12)

#### 4.2.2 Imaging Encoder — 3D ResNet-50

- Library: MONAI (Medical Open Network for AI) — [monai.io](https://monai.io)
- Architecture: ResNet-50 with spatial\_dims=3, n\_input\_channels=1, output 256-dim embedding
- Pre-training: Transfer learning from Med3D weights (pre-trained on 8 medical segmentation datasets)
- Fine-tuning: On HD MRI with cosine LR scheduler, mixed precision (torch.cuda.amp), early stopping patience=10
- Augmentation: Random flip (all axes), random rotation ( $\pm 15^\circ$ ), random intensity shift ( $\pm 0.1$ )

#### 4.2.3 Clinical Encoder — Bi-LSTM

- Input: Sequence of visit feature vectors (CAG repeat, UHDRS motor, UHDRS cognitive, TFC, age at visit)
- Architecture: 2-layer Bi-LSTM, hidden\_size=256, dropout=0.3
- Output: 256-dim embedding from the final timestep (last visit)
- Handles variable-length sequences via PyTorch pack\_padded\_sequence

#### 4.2.4 Fusion Module — Cross-Modal Attention

- Architecture: nn.MultiheadAttention with 8 heads, embed\_dim=256
- Query: imaging embedding; Key/Value: clinical embedding
- Output: 256-dim fused representation passed through LayerNorm + FFN (GELU activation)
- Rationale: Imaging features are weighted by clinical context, not simply concatenated

#### 4.2.5 Disease Stage Classifier

- Architecture: Linear(256, 3) classification head with softmax
- Classes: 0=Pre-manifest, 1=Early HD, 2=Advanced HD
- Loss: Cross-entropy with class weighting inversely proportional to class frequency

#### 4.2.6 Progression Forecaster

- Architecture: Regression head Linear(256, 2) — predicts UHDRS motor score change at 12 and 24 months
- Loss: Huber loss (robust to outliers in longitudinal prediction)
- Output: Continuous score delta + risk category (low/medium/high based on thresholds)

#### 4.2.7 Explainability Module

- GradCAM++: Applied to the final convolutional layer of the 3D ResNet; output is a 3D activation map resized to input volume dimensions; visualised as colour overlay on the 5 central axial slices

- SHAP DeepExplainer: Applied to the clinical encoder; produces per-feature importance scores for CAG repeat, UHDRS subscores, TFC; displayed as a waterfall chart

## 5. Functional Requirements

### 5.1 Data Ingestion

- FR-01: System shall accept NIfTI (.nii or .nii.gz) MRI files via file upload
- FR-02: System shall accept tabular clinical data as JSON or CSV
- FR-03: System shall validate input files and return structured error messages for invalid formats
- FR-04: System shall preprocess inputs automatically without manual intervention

### 5.2 Prediction

- FR-05: System shall classify each patient into one of three HD stages with a confidence score
- FR-06: System shall return a 12-month and 24-month progression risk score
- FR-07: System shall generate a GradCAM++ heatmap as a downloadable PNG
- FR-08: System shall return SHAP feature importance for the top 5 clinical features

### 5.3 Dashboard

- FR-09: Dashboard shall display the predicted disease stage with confidence percentage
- FR-10: Dashboard shall render GradCAM++ heatmaps overlaid on MRI axial slices
- FR-11: Dashboard shall display a SHAP waterfall chart for clinical features
- FR-12: Dashboard shall display a 24-month progression trajectory chart
- FR-13: Dashboard shall allow the clinician to download a PDF report of the full analysis

### 5.4 Ablation Studies (Research Requirement)

- FR-14: System shall support single-modality inference mode (MRI only, clinical only, genetic only)
- FR-15: All ablation results shall be logged to a CSV with model config, split, and metrics

## 6. Non-Functional Requirements

| ID     | Category    | Requirement                                                                                |
|--------|-------------|--------------------------------------------------------------------------------------------|
| NFR-01 | Performance | Full inference (MRI + clinical) in $\leq 30$ seconds on NVIDIA GPU                         |
| NFR-02 | Reliability | Model must produce identical outputs given identical inputs (deterministic inference mode) |
| NFR-03 | Scalability | Architecture shall support addition of new modalities without rewriting core fusion logic  |
| NFR-04 | Security    | Patient data shall never leave the local server; no external API calls with PHI            |

|        |                 |                                                                                                    |
|--------|-----------------|----------------------------------------------------------------------------------------------------|
| NFR-05 | Usability       | A clinician with no ML background shall be able to upload data and read results in under 5 minutes |
| NFR-06 | Maintainability | All modules versioned via Git; model weights stored with hash verification                         |
| NFR-07 | Portability     | System deployable on Ubuntu 22.04 + Python 3.10 + CUDA 11.8 with a single setup script             |

## 7. Technology Stack

| Layer               | Technology               | Justification                                                                         |
|---------------------|--------------------------|---------------------------------------------------------------------------------------|
| Deep Learning       | PyTorch 2.x + MONAI      | MONAI provides production-grade 3D medical imaging transforms and pre-trained ResNets |
| Neuroimaging        | FSL, ANTs, NiBabel       | Industry-standard tools for skull stripping, registration, and NIfTI I/O              |
| Explainability      | pytorch-grad-cam, SHAP   | GradCAM++ for 3D spatial heatmaps; SHAP for tabular feature attribution               |
| Backend API         | FastAPI + Uvicorn        | Async, auto-generates OpenAPI docs, Pydantic validation                               |
| Frontend            | React + Recharts         | Recharts for progression trajectory charts; React for component architecture          |
| Database            | SQLite / PostgreSQL      | SQLite for dev; PostgreSQL for production deployment                                  |
| Experiment Tracking | Weights & Biases (wandb) | Free for academic use; logs loss curves, AUC, confusion matrices automatically        |
| Version Control     | Git + GitHub             | Branch-per-module strategy; main branch requires passing tests                        |
| Environment         | Conda + requirements.txt | Reproducible environment; single conda env for all team members                       |

## 8. Implementation Plan

| Phase | Weeks | Deliverable                                                                         | Owner            |
|-------|-------|-------------------------------------------------------------------------------------|------------------|
| 1     | 1–2   | Dataset access, conda env setup, EDA notebook, class distribution plots             | All members      |
| 2     | 3–4   | MRI preprocessing pipeline (FSL + MONAI transforms), DataLoader class               | Shivam           |
| 3     | 5–6   | 3D ResNet-50 trained, AUC $\geq 0.75$ on val split, training curves logged to wandb | Poorna           |
| 4     | 5–7   | Bi-LSTM clinical encoder trained on longitudinal sequences                          | Prithvi          |
| 5     | 7–8   | Cross-modal attention fusion + end-to-end joint training                            | Poorna + Prithvi |
| 6     | 9     | GradCAM++ visualisation + SHAP summary plots for all test cases                     | Shivam           |

|   |    |                                                                              |             |
|---|----|------------------------------------------------------------------------------|-------------|
| 7 | 10 | FastAPI backend: /predict endpoint with Pydantic validation                  | Suraj       |
| 8 | 11 | React dashboard: stage card, MRI heatmap viewer, SHAP chart, trajectory plot | Suraj       |
| 9 | 12 | Ablation study results, final metrics table, report write-up, demo video     | All members |

## 9. API Specification

### POST /predict

Accepts a multipart form submission. Returns a structured JSON prediction object.

#### Request (multipart/form-data)

| Field           | Type  | Required | Description                                                         |
|-----------------|-------|----------|---------------------------------------------------------------------|
| mri_file        | File  | Yes      | NIfTI file (.nii or .nii.gz), structural T1-weighted MRI            |
| cag_repeat      | Float | Yes      | CAG repeat count from genetic report (range: 36–120)                |
| uhdrs_motor     | Float | Yes      | UHDRS Total Motor Score (0–124)                                     |
| uhdrs_cognitive | Float | Yes      | UHDRS Cognitive Assessment Battery score                            |
| tfc_score       | Float | No       | Total Functional Capacity score (0–13); defaults to null if omitted |
| age             | Float | Yes      | Patient age in years at time of assessment                          |

#### Response (application/json)

| Field               | Type   | Description                                                           |
|---------------------|--------|-----------------------------------------------------------------------|
| stage               | String | Predicted disease stage: "pre-manifest"   "early"   "advanced"        |
| confidence          | Float  | Softmax probability for the predicted class (0.0–1.0)                 |
| stage_probabilities | Object | {pre_manifest: float, early: float, advanced: float} — all sum to 1.0 |
| progression_12mo    | Float  | Predicted UHDRS motor score change at 12 months                       |
| progression_24mo    | Float  | Predicted UHDRS motor score change at 24 months                       |
| risk_category       | String | Progression risk: "low"   "medium"   "high"                           |
| gradcam_url         | String | URL to downloadable GradCAM++ heatmap PNG (5 axial slices)            |
| shap_features       | Array  | Top 5 SHAP feature importances: [{name, value, impact}]               |
| processing_time_s   | Float  | End-to-end inference time in seconds                                  |

## 10. Prompting Guide for Claude (Opus)

This section provides the exact prompt templates to use when building NeuroSense with a Claude-powered coding assistant. Each prompt is scoped to a single module for maximum reliability.

### Master Prompt Template

Use this structure for every prompt:

- **ROLE:** You are a senior ML engineer building a medical AI system called NeuroSense.
- **CONTEXT:** [paste the relevant section from this PRD]
- **TASK:** [one specific, bounded coding task]
- **CONSTRAINTS:** [library versions, output format, what NOT to do]
- **DELIVERABLE:** [exact filename and class/function names expected]

### Phase-by-Phase Prompt Templates

#### Phase 1: MONAI Dataset Class

Use this prompt to generate the PyTorch Dataset:

*"You are a senior ML engineer. Build a PyTorch Dataset class called HuntingtonDataset for PREDICT-HD data in BIDS format. Each `__getitem__` must return a dict: {mri: Tensor[1,96,96,96], clinical: Tensor[5], label: int, subject\_id: str}. The clinical vector contains [cag\_repeat, uhdrs\_motor, uhdrs\_cognitive, tfc, age] all normalised to [0,1]. Use MONAI LoadImage for MRI loading. Include a static method `split(root_dir, seed=42)` returning train/val/test DataLoaders with subject-level stratification by label. Add full type hints and docstrings."*

#### Phase 2: 3D ResNet Encoder

*"Using MONAI's ResNet class, build a class MRIEncoder that wraps a 3D ResNet-50 (spatial\_dims=3, n\_input\_channels=1). Remove the final classification layer and add a projection head: `Linear(2048, 256) → ReLU → LayerNorm` to produce a 256-dim embedding. Add a separate ClassificationHead class: `Linear(256, 3)`. Write a training loop function `train_mri_encoder(model, head, train_loader, val_loader, epochs=50)` that uses cosine LR warmup (5 epochs) then cosine decay, AdamW optimiser, mixed precision via `torch.cuda.amp.GradScaler`, and logs per-class AUC-ROC each epoch using sklearn. Include early stopping with `patience=10`."*

#### Phase 3: Bi-LSTM Clinical Encoder

*"Build a class ClinicalEncoder in PyTorch. Input is a padded sequence of visit feature vectors (batch, max\_visits, 5). Use 2-layer Bi-LSTM with hidden\_size=256, dropout=0.3. Use `pack_padded_sequence` and `pad_packed_sequence` to handle variable-length sequences. Output the hidden state at the last valid timestep for each sample as a (batch, 512) tensor. Add a projection layer `Linear(512, 256)`. Include a forward method signature: `forward(self, x: Tensor, lengths: Tensor) -> Tensor`."*

#### Phase 4: Cross-Modal Fusion

*"Build a class CrossModalFusion in PyTorch. It accepts `img_emb: Tensor[B,256]` and `clin_emb: Tensor[B,256]`. Use `nn.MultiheadAttention(embed_dim=256, num_heads=8, batch_first=True)`*

where `img_emb` is the query and `clin_emb` is both key and value. Apply a pre-norm transformer block: `LayerNorm` → `attention` → `residual` → `LayerNorm` → `FFN(256→1024→256, GELU)` → `residual`. Output shape: `(B, 256)`. Add a `NeuroSenseModel` class that composes `MRIEncoder` + `ClinicalEncoder` + `CrossModalFusion` + `ClassificationHead` + `ProgressionHead(Linear(256,2))` in a single forward pass."

### **Phase 5: GradCAM++ Explainability**

"Using the `pytorch-grad-cam` library, write a function `explain_mri(model: MRIEncoder, mri_tensor: Tensor, pred_class: int) -> np.ndarray` that applies GradCAMPlusPlus to `model.layer4[-1]` and returns a 3D numpy activation map of the same spatial shape as the input. Then write `plot_axial_heatmaps(mri: np.ndarray, cam: np.ndarray, n_slices: int = 5, save_path: str)` that selects the 5 central axial slices, overlays the CAM using a jet colormap at `alpha=0.5`, and saves as a PNG. Do not use `matplotlib.show()` — only save to file."

### **Phase 6: FastAPI Backend**

"Build a FastAPI app in `app/main.py`. Define a POST `/predict` endpoint using multipart form data. Use Pydantic to validate: `mri_file` (`UploadFile`, must be `.nii` or `.nii.gz`), `cag_repeat` (`float`, 36–120), `uhdrs_motor` (`float`, 0–124), `uhdrs_cognitive` (`float`, `ge=0`), `age` (`float`, 18–90). Load the `NeuroSenseModel` from a checkpoint at startup using an `@app.on_event('startup')` handler. Run preprocessing, inference, GradCAM++, and SHAP inside the endpoint. Return a `PredictionResponse` Pydantic model with all fields from Section 9 of this PRD. Add `/health` and `/version` endpoints. Use `uvicorn` for serving."

## 11. Ablation Study Design

The ablation study is the most important section of the final report because it directly proves that multi-modal fusion adds measurable value over any single modality.

| Condition                | MRI | Clinical     | Expected AUC    |
|--------------------------|-----|--------------|-----------------|
| MRI only (baseline)      | ✓   | ✗            | ~0.78–0.82      |
| Clinical only            | ✗   | ✓            | ~0.74–0.78      |
| Genetic only (CAG)       | ✗   | ✗ (CAG only) | ~0.70–0.75      |
| MRI + Clinical (concat)  | ✓   | ✓            | ~0.82–0.85      |
| Full NeuroSense (fusion) | ✓   | ✓            | ≥ 0.87 (target) |

Log all ablation results to `ablation_results.csv` with columns: `[run_id, condition, seed, auc_premanifest, auc_early, auc_advanced, auc_mean, fl_macro, val_loss]`. Run each condition with 3 different random seeds and report mean  $\pm$  std.

## 12. Risks and Mitigations

| Risk                                                   | Severity | Mitigation                                                                                                             |
|--------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------|
| PREDICT-HD access delayed beyond 4 weeks               | High     | Begin development on OpenNeuro ds004040 immediately; structure code so dataset path is configurable                    |
| Insufficient GPU memory for 3D ResNet                  | Medium   | Use gradient checkpointing, reduce batch size to 2, use mixed precision (fp16)                                         |
| Class imbalance (pre-manifest >> advanced)             | High     | Use inverse frequency class weighting in cross-entropy loss; report per-class AUC not overall accuracy                 |
| Small effective dataset size for 3D training           | High     | Transfer learning from Med3D weights; aggressive augmentation; start with 2D ResNet as a sanity baseline               |
| Multi-modal fusion does not outperform single modality | Medium   | Try concatenation fusion as a baseline before cross-attention; report both to show progression                         |
| GradCAM++ produces uninformative maps                  | Low      | Fall back to Integrated Gradients (also in pytorch-grad-cam); validate on known HD-affected regions (caudate, putamen) |

## 13. Recommended Repository Structure

Organise the codebase as follows from day one. This makes it easy for each team member to work on their module independently and ensures the AI coding assistant knows where to place files.

- **neurosense/**
  - **data/** — Dataset classes, preprocessing scripts, BIDS loaders
  - **models/** — `mri_encoder.py`, `clinical_encoder.py`, `fusion.py`, `classifier.py`
  - **training/** — `train.py`, `ablation.py`, `evaluate.py`, `losses.py`
  - **explainability/** — `gradcam.py`, `shap_analysis.py`, `visualise.py`
  - **api/** — `main.py` (FastAPI), `schemas.py` (Pydantic), `inference.py`
  - **frontend/** — React app (`src/`, `public/`, `package.json`)
  - **notebooks/** — `EDA.ipynb`, `ablation_results.ipynb`
  - **configs/** — `model_config.yaml`, `train_config.yaml`
  - **scripts/** — `setup_env.sh`, `download_data.sh`, `preprocess_all.sh`
  - **tests/** — `test_dataset.py`, `test_models.py`, `test_api.py`
  - `requirements.txt`, `environment.yml`, `README.md`, `.gitignore`

## 14. Acceptance Criteria for Project Evaluation

The following criteria must all be met for the project to be considered complete for evaluation:

1. All five model modules are implemented (encoder, clinical encoder, fusion, classifier, forecaster)
2. Mean AUC-ROC  $\geq 0.82$  on the held-out test split (reported with per-class breakdown)
3. Ablation study demonstrates that multi-modal fusion outperforms all single-modality baselines
4. GradCAM++ heatmaps are generated and highlight neurologically plausible brain regions (caudate, striatum)
5. SHAP analysis shows CAG repeat count as the highest-importance feature (expected from literature)
6. FastAPI /predict endpoint returns correct response schema within 30 seconds on GPU
7. React dashboard renders all outputs: stage card, heatmap, SHAP chart, trajectory plot
8. All code is version-controlled on GitHub with a README that includes setup instructions
9. A 5-minute live demo can be run end-to-end from the React UI without manual terminal commands